

Software Requirements Specification

Vehicle Parking System

Group 14 — Problem Statement No. 10

Project	Vehicle Parking System (MERN Stack)
Date	06-09-2026
Authors	Siddhesh Dattatray Koditkar — PES2UG24CS502 Sumukh M Gowda — PES2UG24CS537 Sparsha Arun — PES2UG24CS513 Sinchana Kulkarni — PES2UG24CS504

Table of Contents

1. Introduction
2. Overall Description
3. External Interface Requirements
4. System Features
5. Non-Functional Requirements
- 5.1 Security
6. Quality Attributes & Acceptance Tests
7. System Models and Diagrams (UML Use-Case Diagrams)
8. Requirements Traceability Matrix (RTM)

1. Introduction

1.1 Purpose

This document is a Software Requirements Specification (SRS) for the Vehicle Parking System (VPS), a MERN-stack (MongoDB, Express, React, Node.js) software system that manages vehicle entry, parking-spot allocation, exit processing, and fare collection at a parking facility with multiple entry and exit gates. It defines functional and non-functional requirements, interfaces, and verification criteria for the development team (including AI coding assistants used during implementation) and course evaluators.

1.2 Scope

The VPS covers: operator-driven vehicle entry and spot allocation across separate two-wheeler (2W) and four-wheeler (4W) zones; concurrency-safe allocation across multiple simultaneous entry gates; exit processing with tiered fare calculation; ticket-ID based and manual (plate-number) lookup at exit; overstay and lost-ticket handling with admin-enforced penalties; an admin-only analytics and audit dashboard; and real-time synchronization of system state across every open client session.

The system is designed to run as a fully self-contained local simulation. Multiple browser tabs, each authenticated as a different user (two entry-gate operators, two exit-gate operators, one admin), stand in for physically separate terminals, all connected to a single Node/Express backend and a single MongoDB instance. The following are explicitly out of scope: physical barrier/gate hardware, camera-based license-plate recognition, real payment-gateway integration, a customer-facing portal or online pre-booking, and deployment across physically separate networked sites.

1.3 Audience

Developers, QA reviewers and the project team (Group 14).

1.4 Definitions, Acronyms and Abbreviations

- VPS — Vehicle Parking System
- FR / NFR — Functional Requirement / Non-Functional Requirement
- 2W / 4W — Two-Wheeler / Four-Wheeler
- RBAC — Role-Based Access Control
- JWT — JSON Web Token
- FCFS — First-Come, First-Served
- ODM — Object Document Mapper (Mongoose)
- RTM — Requirements Traceability Matrix
- UML — Unified Modeling Language

2. Overall Description

2.1 Product Perspective

The VPS is a self-contained web application: a single Express/Node.js backend exposing a REST API and a Socket.io real-time channel, a single MongoDB database (via Mongoose), and a React single-page frontend. Rather than physically separate terminals, the system is demonstrated using multiple browser tabs on one machine, each independently authenticated, all connected live to the same backend — so an action performed in one tab (e.g., a spot being allotted at Entry-1) is reflected instantly in every other open tab (Entry-2, both exit consoles, and the Admin dashboard).

2.2 Major Product Functions

- Gate-scoped operator authentication and admin authentication
- Vehicle entry registration and zone-aware (2W/4W) spot allocation

- Concurrency-safe, FCFS-ordered spot allocation across multiple entry gates
- Ticket-ID generation at entry and fast lookup at exit
- Real-time, colour-coded (green/red) parking-lot visualization across all sessions
- Exit processing with exact-duration, tiered fare calculation (separate rate tables for 2W/4W)
- Lost-ticket handling via manual lookup plus penalty
- Overstay detection, operator-to-admin notification, and admin-enforced force-release with penalty
- Admin-only revenue and occupancy analytics dashboard
- Admin-only access to full raw transaction/audit records
- A separate, out-of-scope developer Simulation Console for demo/testing (see 2.6)

2.3 User Roles and Characteristics

The system recognises exactly two real system roles. Both authenticate the same way (credentialed login issuing a JWT) but are authorized very differently server-side.

Role	Characteristics & Access
Gate Operator	Logs in with credentials scoped to exactly one physical gate (Entry-1, Entry-2, Exit-1, or Exit-2). Handles vehicle intake/spot allocation (entry gates) or fare calculation/exit (exit gates). Cannot view raw historical database records, analytics, or other gates' queues.
Admin / Manager	Logs in with a single root-access account. Views live occupancy, revenue and occupancy analytics, and the full transaction/audit log. Receives overstay notifications from operators and is the only role permitted to force-release a spot or configure fare/spot settings.

2.4 Operating Environment

- Backend: Node.js + Express, Mongoose ODM, Socket.io for real-time broadcast
- Database: MongoDB (local instance or MongoDB Atlas)
- Frontend: React SPA, served to any modern desktop browser
- Demonstration environment: multiple browser tabs/windows on a single host machine, each an independent authenticated session

2.5 Design and Implementation Constraints

- No physical hardware integration (no real barrier, sensor, or camera)
- No real payment gateway; fare is calculated and displayed, not processed as a live transaction
- No customer-facing interface or online pre-booking
- All concurrency control must be enforced at the database layer, since multiple tabs may submit near-simultaneous requests to the same backend instance
- Fare-tier and spot-zone configuration must be stored as data, not hard-coded, so the Admin can adjust them without redeploying code

2.6 Assumptions and Testing/Demonstration Support

For demonstration and load-testing purposes, a separate Simulation Console is provided. It is a developer/testing utility only — it is not a system role, is not reachable through the Gate Operator or Admin login flows, and has no entry in the functional role model in 2.3. It allows a developer to generate synthetic vehicle arrivals at an adjustable rate and to override fare parameters at runtime, purely so that a live demo does not depend on manually typing plate numbers into four separate consoles. Requirements for this console are listed separately in section 4.9 and are excluded from the acceptance criteria that govern the deployed system.

3. External Interface Requirements

3.1 User Interfaces

- Entry Console (Entry-1 / Entry-2): a queue of arriving vehicles, a form to capture plate number and vehicle type, and an “Allot Spot” action that displays the assigned spot and generated Ticket ID.
- Exit Console (Exit-1 / Exit-2): a lookup field (Ticket ID or plate number), a fare breakdown display, and a “Confirm Exit” action.
- Admin Dashboard: live parking-lot map, revenue/occupancy charts, full transaction log with search, and overstay notifications with a force-release control.
- Shared live lot-map visual: green/red colour-coded spots split into 2W and 4W zones, rendered in a minimalist red-and-white theme with black accents and no decorative animation.

3.2 Hardware Interfaces

None. The system is fully software-simulated; no physical card reader, barrier, sensor, or camera is integrated. Any reference to “gates” refers to a logical operator terminal, not physical equipment.

3.3 Software Interfaces

- MongoDB via Mongoose ODM — persistent storage for spots, vehicles, transactions, users, and audit logs
- Socket.io — real-time, bidirectional broadcast of state changes (spot status, notifications) to all connected clients
- Express REST API — CRUD and action endpoints consumed by the React frontend
- JWT-based authentication middleware — validates every protected API and socket connection

3.4 Communications

- WebSocket (Socket.io) channel for live state propagation between server and all open clients
- HTTPS/TLS 1.2+ is the required transport for any non-local deployment; the local demo may run over plain HTTP on localhost only

4. System Features (Detailed)

Each requirement below includes acceptance criteria and a reference test case. IDs follow VPS-F-###, grouped by feature area in blocks of ten so related requirements stay numerically close. Section 4.9 lists Simulation Console requirements separately (prefix VPS-DEV-###) since this utility is not part of the deployed system's role model (see 2.6).

4.1 Authentication & Access Control

Description: authenticate Gate Operators against a single bound gate, and the Admin against a root account, using JWT-based sessions.

Req ID	Requirement (shall...)	Type	Priority	Source / Stakeholder	Acceptance Criteria / Test Ref	Deps
VPS-F-001	The system shall authenticate an operator login only against the single gate (Entry-1, Entry-2, Exit-1, or Exit-2) that account is bound to.	Functional	High	Security / Course Req.	AC: login with Entry-1 credentials on the Exit-1 console is rejected. Test: TC-Auth-01	—
VPS-F-002	The system shall authenticate the Admin via a single root-access account distinct from all operator accounts.	Functional	High	Security	AC: Admin login exposes admin-only routes; operator logins cannot reach them. Test: TC-Auth-02	—
VPS-F-003	The system shall lock an account for a configurable cooldown period after 5	Functional	Medium	Security	AC: 5th consecutive failure triggers logout + audit entry.	VPS-F-060

Req ID	Requirement (shall...)	Type	Priority	Source / Stakeholder	Acceptance Criteria / Test Ref	Deps
	consecutive failed login attempts and record an audit event.				Test: TC-Auth-03	
VPS-F-004	The system shall mask password entry on every login form and shall not log raw password characters.	Functional	High	Security / PCI-style	AC: password field renders masked; server logs contain no plaintext password. Test: TC-Auth-04	—
VPS-F-005	The system shall issue a signed JWT session token on successful login and reject any protected API or socket request lacking a valid, unexpired token.	Functional	High	Security	AC: request without token returns 401; expired token is rejected. Test: TC-Auth-05	—

4.2 Vehicle Entry & Spot Allocation

Description: capture arriving-vehicle details and allocate a spot atomically and fairly across multiple entry gates.

Req ID	Requirement (shall...)	Type	Priority	Source / Stakeholder	Acceptance Criteria / Test Ref	Deps
VPS-F-010	The system shall allow an Entry-gate operator to record incoming vehicle details: plate number, vehicle type (2W/4W), entry gate ID, and entry timestamp.	Functional	High	Gate Operator	AC: record is created with all fields populated server-side. Test: TC-Entry-01	—
VPS-F-011	The system shall query for an available spot only within the zone matching the vehicle's declared type (2W or 4W).	Functional	High	Business Rule	AC: a 4W vehicle is never allotted a 2W-zone spot and vice versa. Test: TC-Entry-02	—
VPS-F-012	The system shall atomically allot the earliest available matching-zone spot using a single atomic database operation (e.g., findOneAndUpdate), such that two concurrent requests can never be allotted the same spot.	Functional	High	Reliability / Concurrency	AC: 50 concurrent allocation requests under load testing produce zero duplicate assignments. Test: TC-Entry-03	VPS-NF-002
VPS-F-013	The system shall process simultaneous allocation requests arriving from multiple entry gates strictly in first-come-first-served order, based on server-received timestamp.	Functional	High	Business Rule	AC: under simultaneous submission, the earlier-received request is served first in 100% of test runs. Test: TC-Entry-04	VPS-F-012
VPS-F-014	The system shall reject entry and display a "Lot Full" message to the operator when no spot is available in the requested zone.	Functional	High	Gate Operator	AC: allocation request against a full zone returns a Lot-Full response, no record is created. Test: TC-Entry-05	—
VPS-F-015	The system shall generate a unique Ticket ID for every successfully allotted vehicle and associate it with that entry record.	Functional	High	Gate Operator	AC: Ticket ID is unique across all active records and persists with the entry. Test: TC-Entry-06	—
VPS-F-016	The system shall display the generated Ticket ID, including a QR-style visual, on the operator's screen immediately after allocation.	Functional	Medium	Gate Operator	AC: Ticket ID and QR-style image render on-screen within 1 second of allocation. Test: TC-Entry-07	VPS-F-015

4.3 Real-Time Lot Visualization

Description: give every open session (both entry consoles, both exit consoles, and the Admin dashboard) an identical, live view of spot occupancy.

Req ID	Requirement (shall...)	Type	Priority	Source / Stakeholder	Acceptance Criteria / Test Ref	Deps
VPS-F-020	The system shall display a visual map of all parking spots, colour-coded green (free) or red (reserved), grouped into 2W and 4W zones.	Functional	High	All Roles	AC: every spot's colour matches its current stored status. Test: TC-Viz-01	—
VPS-F-021	The system shall broadcast any spot-status change to all connected clients via WebSocket within 2 seconds, with no page refresh required.	Functional	High	Reliability / UX	AC: status change in one session is visible in another open session within 2s. Test: TC-Viz-02	VPS-NF-001
VPS-F-022	The system shall keep the lot visualization identical and synchronized across every currently open session at all times.	Functional	High	All Roles	AC: two simultaneously open sessions never show conflicting spot states. Test: TC-Viz-03	VPS-F-021
VPS-F-023	The system shall render the visualization in a minimalist red-and-white theme with black accents, without decorative animation.	Functional	Low	UX / Stakeholder pref.	AC: visual review against the agreed style reference. Test: TC-Viz-04	—

4.4 Vehicle Exit & Fare Calculation

Description: locate the correct entry record at exit and compute an exact, tiered, vehicle-type-specific fare.

Req ID	Requirement (shall...)	Type	Priority	Source / Stakeholder	Acceptance Criteria / Test Ref	Deps
VPS-F-030	The system shall allow an Exit-gate operator to retrieve a vehicle's entry record by Ticket ID.	Functional	High	Gate Operator	AC: valid Ticket ID returns the exact matching entry record. Test: TC-Exit-01	VPS-F-015
VPS-F-031	The system shall allow an Exit-gate operator to retrieve a vehicle's entry record via manual plate-number search when the Ticket ID is unavailable.	Functional	High	Gate Operator	AC: searching by plate number returns the correct active entry record. Test: TC-Exit-02	—
VPS-F-032	The system shall compute parking duration as the exact difference, down to the minute, between recorded entry and exit timestamps.	Functional	High	Business Rule	AC: computed duration matches wall-clock difference to the minute. Test: TC-Exit-03	—
VPS-F-033	The system shall apply a tiered fare structure — the first hour free, then a base hourly rate that increases at defined thresholds (e.g., every 2–3 hours) — computed on the exact fractional duration with no rounding.	Functional	High	Business Rule	AC: fare for a sample set of durations matches the configured tier table to the exact sub-unit. Test: TC-Exit-04	VPS-F-032
VPS-F-034	The system shall maintain independently configurable fare-tier tables for two-wheelers and four-wheelers.	Functional	High	Business Rule	AC: identical duration produces different fares for 2W vs 4W per configured tables. Test: TC-Exit-05	VPS-F-033
VPS-F-035	The system shall apply a lost-ticket penalty, in addition to the computed fare, whenever a vehicle exits without a valid Ticket ID being presented.	Functional	High	Business Rule	AC: exit via plate-number search alone adds the configured penalty to the final fare. Test: TC-Exit-06	VPS-F-031
VPS-F-036	The system shall mark the vehicle's spot as free, in real time, immediately upon fare confirmation at exit.	Functional	High	Gate Operator	AC: spot colour changes to green in all sessions within 2s of exit confirmation. Test: TC-Exit-07	VPS-F-021
VPS-F-037	The system shall permanently record the finalized transaction (plate, entry/exit time, duration, fare, gate IDs, operator ID) in MongoDB.	Functional	High	Admin / Audit	AC: a queryable transaction record exists after every completed exit. Test: TC-Exit-08	VPS-F-060

4.5 Overstay & Lost-Ticket Escalation

Description: detect vehicles that overstay a configurable threshold and route resolution through the Admin.

Req ID	Requirement (shall...)	Type	Priority	Source / Stakeholder	Acceptance Criteria / Test Ref	Deps
VPS-F-040	The system shall flag a parked vehicle as "overstayed" once its parked duration exceeds a configurable threshold.	Functional	Medium	Business Rule	AC: flag appears automatically once threshold is crossed, without operator action. Test: TC-Over-01	—
VPS-F-041	The system shall allow a Gate Operator to send an overstay notification to the Admin, including the vehicle's ticket/plate and current duration.	Functional	High	Gate Operator	AC: notification appears on the Admin dashboard within 2s of being sent. Test: TC-Over-02	VPS-F-021
VPS-F-042	The system shall permit only the Admin role to force-release an overstayed vehicle's spot.	Functional	High	Security / Business Rule	AC: force-release action from a non-admin account is rejected server-side. Test: TC-Over-03	VPS-F-050
VPS-F-043	The system shall automatically apply an overstay penalty to the final fare whenever a force-release occurs.	Functional	High	Business Rule	AC: force-released transaction's fare includes the configured overstay penalty. Test: TC-Over-04	VPS-F-042

4.6 Admin Analytics & Oversight

Description: give the Admin visibility and controls that Gate Operators do not have.

Req ID	Requirement (shall...)	Type	Priority	Source / Stakeholder	Acceptance Criteria / Test Ref	Deps
VPS-F-050	The system shall restrict access to the analytics dashboard and raw transaction database to the Admin role only, enforced server-side.	Functional	High	Security	AC: an operator account calling admin endpoints receives 403. Test: TC-Admin-01	PRJ-SR-005
VPS-F-051	The system shall display real-time total and per-zone (2W/4W) occupancy percentages to the Admin.	Functional	Medium	Admin	AC: displayed percentage matches (reserved / total) per zone at query time. Test: TC-Admin-02	—
VPS-F-052	The system shall display a revenue-over-time chart to the Admin, computed from recorded transactions.	Functional	Medium	Admin	AC: chart totals reconcile with the sum of transaction fares for the selected period. Test: TC-Admin-03	VPS-F-037
VPS-F-053	The system shall display an occupancy/busyness-over-time chart to the Admin, highlighting peak usage periods.	Functional	Medium	Admin	AC: chart correctly reflects historical occupancy snapshots. Test: TC-Admin-04	—
VPS-F-054	The system shall allow the Admin to search and view full transaction records (plate, timestamps, gate, operator, fare) that Gate Operators cannot view.	Functional	High	Admin / Security	AC: search returns matching records to Admin; identical request from an operator account is rejected. Test: TC-Admin-05	VPS-F-050
VPS-F-055	The system shall allow the Admin to configure fare-tier thresholds and rates independently for each vehicle type.	Functional	Medium	Admin	AC: updated tier values apply to subsequent fare calculations without a redeploy. Test: TC-Admin-06	VPS-F-034
VPS-F-056	The system shall allow the Admin to configure the total number of spots and the 2W/4W zone split.	Functional	Low	Admin	AC: updated spot counts are reflected in the lot visualization for all sessions. Test: TC-Admin-07	VPS-F-020

4.7 Audit Logging & Data Integrity

Description: make every state-changing action attributable and prevent unauthorized data access.

Req ID	Requirement (shall...)	Type	Priority	Source / Stakeholder	Acceptance Criteria / Test Ref	Deps
VPS-F-060	The system shall write an append-only audit-log entry for every spot allocation, release, force-release, and penalty application, recording the acting user, timestamp, and action.	Functional	High	Security / Compliance	AC: every such action produces exactly one corresponding audit entry. Test: TC-Audit-01	—
VPS-F-061	The system shall prevent Gate Operator accounts from reading, editing, or deleting audit-log entries.	Functional	High	Security	AC: operator-authenticated requests to audit endpoints are rejected server-side. Test: TC-Audit-02	VPS-F-060
VPS-F-062	The system shall reject any client attempt to modify spot status or fare data outside the defined, validated API endpoints.	Functional	High	Security	AC: direct/malformed write attempts are rejected and logged. Test: TC-Audit-03	VPS-F-060

4.9 Simulation Console (Out-of-Scope Testing/Demo Utility)

The following requirements describe a developer-only utility used for demonstrations and load testing. It is intentionally excluded from the role model in 2.3 and from the acceptance criteria in Section 6 that govern the deployed system.

Req ID	Requirement (shall...)	Type	Priority	Source / Stakeholder	Acceptance Criteria / Test Ref	Deps
VPS-DEV-001	The Simulation Console shall generate synthetic vehicle arrivals at a configurable rate, distributed across the available entry gates.	Functional (dev-only)	Low	Development Team	AC: enabling the console produces plausible entry traffic without manual input. Test: TC-Dev-01	—
VPS-DEV-002	The Simulation Console shall allow runtime adjustment of traffic-generation rate and fare-tier parameters for demonstration purposes.	Functional (dev-only)	Low	Development Team	AC: adjusting a control changes system behaviour without a restart. Test: TC-Dev-02	VPS-DEV-001
VPS-DEV-003	The Simulation Console shall not be reachable via the Gate Operator or Admin login flows and shall be excluded from the deployed feature scope.	Functional (dev-only)	Medium	Development Team	AC: no navigation path to the console exists from either role's UI. Test: TC-Dev-03	—

5. Non-Functional Requirements (Detailed)

NFRs below are measurable and tied to test plans. IDs follow VPS-NF-###.

Req ID	Requirement (shall...)	Category	Priority	Acceptance Criteria / Measurement
VPS-NF-001	Real-time spot-status updates shall propagate to all connected clients within 2 seconds under normal load.	Performance	High	Measured latency $\leq 2s$ for 95% of updates in test. Test: TC-Perf-01
VPS-NF-002	Concurrent allocation requests from multiple gates shall never result in the same spot being assigned to two vehicles.	Reliability / Concurrency	High	Zero duplicate allocations across ≥ 50 simultaneous requests in load test. Test: TC-Rel-01
VPS-NF-003	The backend shall recover from a restart without losing spot-state or transaction data.	Availability	High	Post-restart state matches pre-restart MongoDB contents. Test: TC-Ops-01
VPS-NF-004	A Gate Operator shall be able to complete an	Usability	Medium	UI walkthrough confirms ≤ 3

Req ID	Requirement (shall...)	Category	Priority	Acceptance Criteria / Measurement
	entry or exit transaction in no more than 3 UI interactions after login (excluding data entry).			interactions. Test: TC-UX-01
VPS-NF-005	Fare-tier rules and spot/zone configuration shall be stored as data, editable by the Admin without code changes.	Maintainability	Medium	Admin edits tier values via UI; change takes effect without redeploy. Test: TC-Maint-01
VPS-NF-006	The data model shall support increasing the number of gates or spots via configuration without architectural changes.	Scalability	Medium	Adding a gate/spot requires only config/data changes, verified by code review. Test: TC-Scale-01
VPS-NF-007	Fare calculations shall be accurate to the nearest currency sub-unit, with no rounding of parking duration.	Accuracy	High	Computed fares match hand-calculated reference values exactly. Test: TC-Acc-01

5.1 Security

5.1.1 Security Objectives

- SO-1: Prevent unauthorized access to vehicle, revenue, and audit data — only the Admin role may view raw transaction records and analytics.
- SO-2: Preserve the integrity of parking-spot state and fare data against tampering, forged requests, and race-condition corruption.

5.1.2 Security Requirements

Req ID	Requirement (shall...)	Type	Priority	Acceptance Criteria / Test Ref
PRJ-SR-001	TLS 1.2+ shall be mandatory for all client-server traffic in any non-local deployment.	Security	High	AC: non-TLS connections are refused in deployed config. Test: TC-Sec-01
PRJ-SR-002	All API endpoints except login shall require a valid JWT; requests without one shall be rejected with 401.	Security	High	AC: unauthenticated request to a protected route returns 401. Test: TC-Sec-02
PRJ-SR-003	Passwords shall be stored only as salted hashes (e.g., bcrypt); plaintext passwords shall never be persisted or logged.	Security	High	AC: database inspection shows only hashed values. Test: TC-Sec-03
PRJ-SR-004	Gate-operator credentials shall be scoped server-side to exactly one gate; actions requested against a different gate shall be rejected regardless of client-side state.	Security	High	AC: server rejects a cross-gate action even if attempted directly via API. Test: TC-Sec-04
PRJ-SR-005	Only the Admin role shall be authorized to access raw database records, analytics endpoints, and force-release actions, enforced server-side.	Security	High	AC: operator-authenticated calls to admin-only endpoints are rejected. Test: TC-Sec-05
PRJ-SR-006	All state-changing requests (allocation, release, force-release, penalty) shall be recorded in an append-only audit log.	Security	Medium	AC: every such request produces a corresponding audit entry. Test: TC-Sec-06

6. Quality Attributes & Acceptance Tests

Exit criteria for acceptance: all High-priority functional requirements implemented and verified, no critical NFR failures, and the RTM (Section 8) shows all listed test cases passed.

Acceptance test suites:

- Authentication & Access Control (VPS-F-001..005, PRJ-SR-001..006)

- Entry & Concurrency-Safe Allocation (VPS-F-010..016, VPS-NF-002)
- Real-Time Visualization (VPS-F-020..023, VPS-NF-001)
- Exit & Fare Calculation (VPS-F-030..037, VPS-NF-007)
- Overstay & Lost-Ticket Escalation (VPS-F-040..043)
- Admin Analytics & Data Access Control (VPS-F-050..056)
- Audit & Integrity (VPS-F-060..062)

7. System Models and Diagrams

7.1 UML Use-Case Diagrams

Two use-case diagrams are provided below, one per system role, covering the functional requirements in Section 4. The out-of-scope Simulation Console (4.9) is intentionally omitted from both, since it is not part of the modeled system.

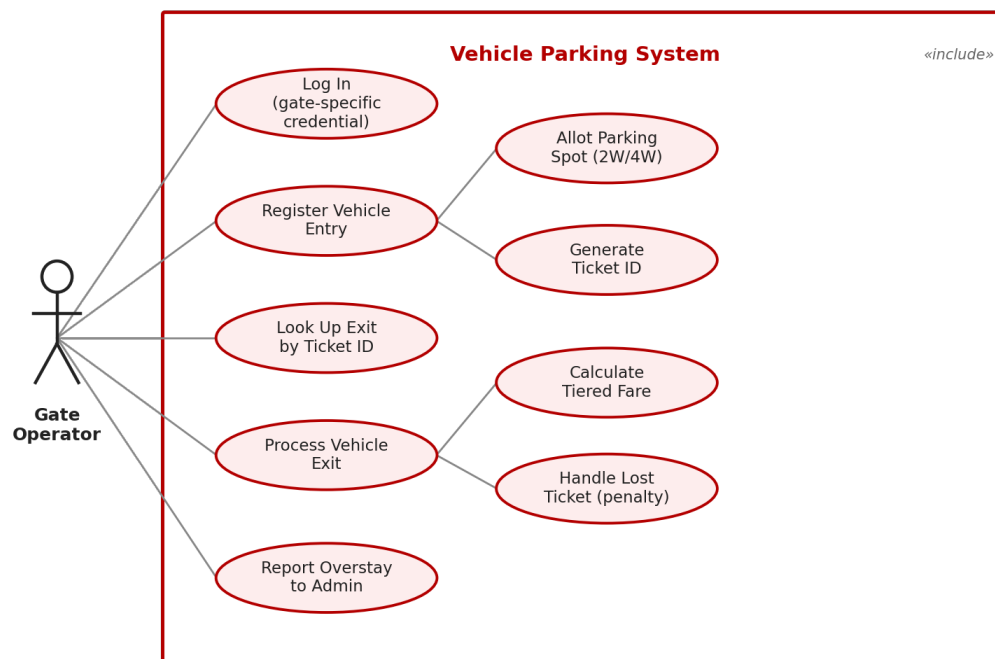


Figure 1 — Gate Operator use-case diagram

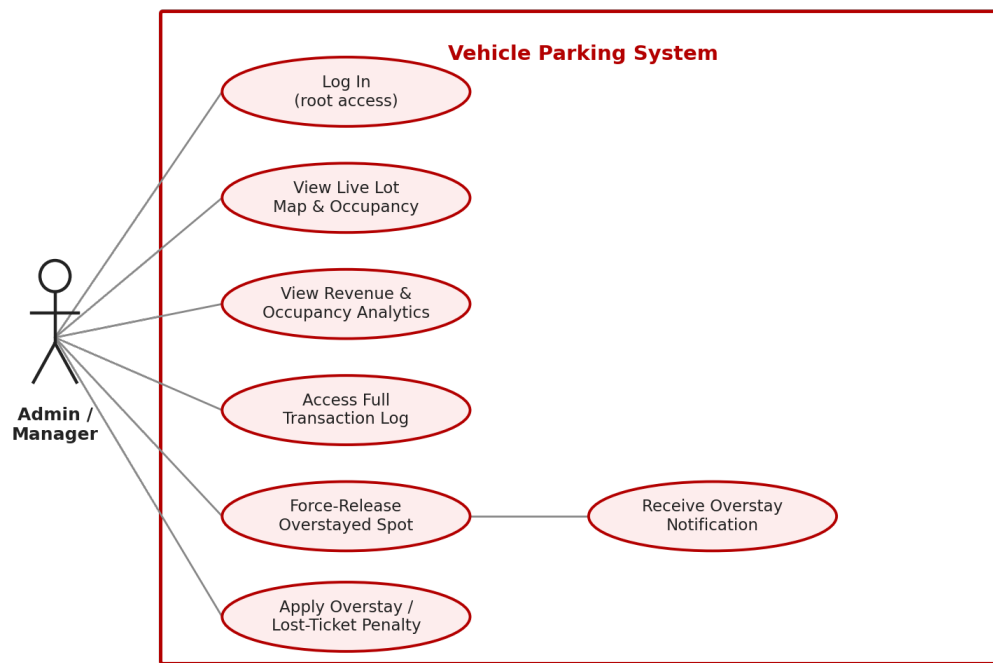


Figure 2 — Admin / Manager use-case diagram

8. Requirements Traceability Matrix (RTM)

Status column uses N (Not started), P (In progress), A (Achieved/passed). All items begin as N at this draft stage.

Req ID	Requirement (short)	Section Ref	Module	Test Case(s)	Status	Comments
VPS-F-001	Gate-scoped operator login	4.1	AuthModule	TC-Auth-01	N	
VPS-F-005	JWT session issuance	4.1	AuthModule	TC-Auth-05	N	
VPS-F-012	Atomic spot allocation	4.2 / VPS-NF-002	AllocationModule	TC-Entry-03	N	Core concurrency req.
VPS-F-013	FCFS multi-gate ordering	4.2	AllocationModule	TC-Entry-04	N	
VPS-F-015	Ticket ID generation	4.2	AllocationModule	TC-Entry-06	N	
VPS-F-021	Real-time broadcast ≤ 2s	4.3 / VPS-NF-001	RealtimeModule (Socket.io)	TC-Viz-02	N	
VPS-F-033	Tiered fare calculation	4.4	FareModule	TC-Exit-04	N	
VPS-F-035	Lost-ticket penalty	4.4	FareModule	TC-Exit-06	N	
VPS-F-042	Admin-only force-release	4.5 / PRJ-SR-005	AdminModule	TC-Over-03	N	
VPS-F-050	Admin-only data access	4.6 / PRJ-SR-005	AdminModule	TC-Admin-01	N	
VPS-F-060	Append-only audit log	4.7	AuditModule	TC-Audit-01	N	
PRJ-SR-003	Password hashing	5.1.2	AuthModule	TC-Sec-03	N	
PRJ-SR-004	Gate-scoped authorization	5.1.2	AuthModule	TC-Sec-04	N	
VPS-NF-002	Zero duplicate allocation under load	5 / 4.2	AllocationModule	TC-Rel-01	N	
VPS-DEV-001	Simulation traffic generator	4.9	SimConsole (dev-only)	TC-Dev-01	N	Excluded from deployed scope